

Implementation Plan

AI-Assisted Return-Fraud Detection E-Commerce Platform (India, COD-focused)

1. Background & Problem Statement

Indian e-commerce sees return fraud rates far above global averages, driven mostly by Cash-on-Delivery (COD) orders, which have almost no verification at the point of purchase. The most common patterns are:

- Wardrobing — buying clothing (often ethnic wear) for a wedding/festival, using it once, then returning it
- Empty-box returns
- Serial over-ordering — ordering multiple sizes/variants with intent to keep one and return the rest

Enterprise fraud-detection tools exist but are priced for large retailers, leaving small-to-mid D2C brands (\$50K–\$5M revenue) with COD checkout exposed and no affordable way to fight this.

2. Solution Summary

A full e-commerce platform — not just a fraud add-on — where fraud prevention is built into the core shopping and returns flow. Every order and return is scored for risk in real time using the customer's own order/return history and passive device signals. High-risk actions require stronger verification before they're allowed to proceed. Low-risk customers experience zero friction.

3. Uniqueness of the Product

- COD-native fraud logic — most fraud tools are built for card/prepaid chargeback fraud; this is designed specifically around India's COD-heavy, low-verification checkout reality.
- Seasonal/cultural signal awareness — ties wardrobing detection to wedding/festival timing, auto-tightening risk thresholds during those windows, a pattern generic global fraud tools don't model.
- Passive-first device signals — lightweight device identification establishes trust without asking the customer to do anything extra.
- Logistics-anchored return proof — trust is anchored in the delivery partner's witnessed pickup rather than customer self-attestation, which is far harder to game.
- Frictionless by default — low-risk customers are auto-approved instantly; step-up verification (SMS OTP, bank-account validation) is reserved for risk-flagged actions only.
- Affordable for small D2C brands — designed for merchants who can't justify enterprise fraud-tool pricing.

4. Two User Personas

Shopper

Register/sign in, browse by category, order (COD or prepaid), track orders, request returns, complete step-up verification when required.

Merchant (admin)

View flagged orders/returns, view customer risk profiles, approve/reject flagged cases, see fraud trends.

5. Core Feature List (do not miss any of these)

Shopper-facing

- Registration page (email/password)
- Google Sign-In
- User profile (personal details, saved addresses, stored credentials such as email)
- Product catalog organized by categories
- Cart and checkout (COD + prepaid)
- Passive device identification captured on every session (no user action required)
- Return request flow:
 - Proof-of-item capture handled at doorstep pickup by the delivery partner (geo-stamped digital signature + item barcode/QR scan), not self-attested by the customer
 - Step-up confirmation (SMS OTP) if the return is flagged medium-risk
- Escalation path if step-up verification fails or is unavailable (see Section 8)
- Visibility into own return count / status (optional transparency to the user)
- Notification (SMS/in-app) to the customer when a return is approved or rejected

Merchant-facing (Admin Dashboard)

- Merchant onboarding — admin account registration, new merchant/store tenant setup, store profile, generation of the merchant-scoped API token used by the dashboard and the browser extension
- Flagged-returns queue (manual review)
- Customer risk profile view (order count, return count, refusal count, risk tier, device trust signals)
- Delivery agent risk view — return rate and flagged-return count per delivery agent/route, surfaced for collusion review
- Approve/reject actions on flagged returns
- Audit log of admin actions — who approved/rejected each flagged case and when, for accountability and to sanity-check self-tuning suggestions
- Fraud trend analytics (returns per week, top flagged customers, category-wise return rates)
- Self-tuning threshold suggestions — dashboard periodically suggests rule-weight adjustments based on outcomes of auto-approved returns vs. flagged returns later cleared on manual review
- Configuration panel for fraud rule weights/thresholds

Fraud Detection (Rule-Based — No ML)

- Risk score computed on every return request and every COD refusal, using:
 - Return frequency per customer
 - COD refusal history
 - Address-mismatch patterns
 - Seasonal signals (e.g., ethnic wear returned shortly after wedding/festival season) — risk thresholds for high-risk categories auto-tighten during known wedding/festival calendar windows rather than staying static year-round
 - Device reuse across accounts/numbers

- Delivery-agent/route risk signal — flags when return fraud clusters around a specific delivery partner or pincode, surfaced as a possible doorstep-collusion signal rather than pure customer-side fraud
- Verification method used (SMS-OTP-confirmed vs bank-validated vs unverified — weighted by confidence)
- Auto-approve low-risk returns instantly
- Flag high-risk returns for manual review + mandatory doorstep proof

Identity & Trust Signals

- Passive device identification — a lightweight, non-invasive device identifier established on every session. Runs automatically; no customer action required.
- SMS OTP confirmation — one-time passcode sent to the registered mobile number for medium-risk order/return actions. Works on any phone with no app or smartphone required.
- Bank-account / UPI VPA validation (penny-drop check) — reserved for high-risk, repeat-offender accounts only; a small verification credit is sent to the customer's bank/UPI account and confirming its receipt ties the account to a real, named bank identity. Combined with forcing the account to prepaid-only checkout, this is the final gate before flagging/blocking an account.
- Doorstep-witnessed return proof — delivery partner captures a geo-stamped digital signature and scans the item's barcode/QR at pickup, replacing customer self-attested return claims.

Every verification event is tagged with `verification_method` so the risk engine weights it by confidence level.

6. Tech Stack

- Frontend: React.js
- Backend: Django + Django REST Framework
- Database: PostgreSQL
- File storage: for doorstep return-proof records (signature captures, scan logs) and device/session telemetry (e.g. S3-compatible storage)
- Device identification: a lightweight, client-generated device token (stored via cookie/local storage), matched server-side for reuse detection — no third-party fingerprinting library required
- SMS OTP: transactional SMS/OTP provider (e.g., MSG91, Twilio SMS, 2Factor) for step-up confirmation on medium-risk actions
- Bank validation: payment gateway's penny-drop / VPA-validation API (e.g., via Razorpay, Cashfree, or a dedicated bank-verification provider), used only for high-risk escalation path
- Festival/wedding calendar data: static regional calendar dataset or calendar API, feeding the seasonal auto-tightening logic (provider TBD — see Open Decisions)
- Customer notifications: SMS/in-app notification on return approval/rejection, reusing the SMS OTP provider's messaging capability where possible
- Multi-tenancy: shared PostgreSQL database with `merchant_id` row-level scoping across all tenant tables (approach TBD — see Open Decisions)

7. Data Model Overview

Merchant (tenant) — `id`, `business_name`, `store_slug`, `plan_tier`, `api_token` (used by admin dashboard and browser extension), `created_at`.

User (shopper or merchant admin, scoped to a merchant/store) — id, merchant_id, email, password_hash, google_id (nullable), role (shopper / merchant_admin), device_token (linked device identification record), profile fields (name, phone, addresses).

Category — id, merchant_id, name.

Product — id, merchant_id, name, category_id, price, stock, images.

Order — id, merchant_id, user_id, order items (see OrderItem below), total, payment_method (COD/prepaid), status, verification_status, verification_method, device_token, created_at.

ReturnRequest — id, order_id, reason, doorstep_proof_file, delivery_agent_id, pincode, verification_status, verification_method, risk_score, status (pending/approved/rejected/manual_review), outcome (auto_approved/flagged_then_cleared/flagged_confirmed_fraud — feeds self-tuning suggestions), reviewed_by (admin user id, nullable), reviewed_at (nullable — audit trail for manual review actions), created_at.

CustomerRiskProfile — merchant_id, user_id, total_orders, total_returns, total_cod_refusals, device_reuse_flag, risk_tier, last_updated.

DeliveryAgentRiskProfile — merchant_id, delivery_agent_id, total_deliveries, total_returns_handled, return_rate, flagged_return_count, risk_flag, last_updated.

All merchant-scoped tables (Category, Product, Order, CustomerRiskProfile, DeliveryAgentRiskProfile) carry merchant_id so one merchant's data is never visible to another. CustomerRiskProfile updates automatically (e.g., via a Django signal) whenever an order or return is created, so the risk score always reflects current behavior.

8. Verification State Machine (Order/Return)

An order or return request is initiated by the shopper. The system first runs passive scoring using device identification and order/return history signals, producing a Low, Medium, or High risk classification.

- If the classification is Low risk, the order or return is auto-approved immediately, with no further steps.
- If the classification is Medium or High risk, step-up verification is triggered: an SMS OTP is sent to the customer's registered mobile number.
 - If the OTP is confirmed, the system proceeds to a risk-scoring pass that incorporates the verification outcome before reaching a final approve/reject decision — verification does not bypass this step.
 - If the OTP is not confirmed or is unavailable, the case escalates along one of two paths depending on context: repeat or high-risk accounts are routed to bank/UPI penny-drop validation plus a prepaid-only restriction going forward; returns are routed to mandatory doorstep-witnessed proof and the manual review queue.

Every verification event is tagged with a verification_method (device_only / sms_otp / bank_validated / doorstep_proof) so the risk engine can weight it by confidence level in later scoring.

Note: seasonal (festival/wedding), delivery-agent/route, and device-reuse signals are not separate decision branches — they are all inputs into the single passive-scoring step described above, which is what produces the Low / Medium / High risk classification.

9. Implementation Phases

Per your request, frontend design comes first, backend follows.

Phase 0 — Design

Wireframes/UI design for: registration, login (Google + email), product listing/category pages, product detail, cart, checkout, order history, return request flow, step-up verification screens (SMS OTP status), admin dashboard.

Phase 1 — Frontend Build (React)

- Auth pages (Google Sign-In + registration/login forms)
- Category and product listing pages
- Product detail, cart, checkout UI
- Passive device identification integration (background script, no UI)
- Step-up verification UI: SMS OTP entry and confirmation status screen
- Order history page
- Return request form (reason, doorstep pickup scheduling — no customer-side proof capture)
- Admin dashboard UI: merchant onboarding/account setup, flagged queue, customer risk profile view, delivery agent risk view, self-tuning suggestions view, analytics

Phase 2 — Backend Build (Django + DRF)

- accounts app: custom user model, Google OAuth integration, profile APIs, merchant onboarding/account setup APIs
- Tenant scoping middleware: enforces merchant_id isolation on every request so one merchant never sees another's data
- catalog app: category/product models and APIs
- orders app: cart, checkout, COD/prepaid order creation, order history APIs
- returns app: return request model, doorstep proof handling, status workflow
- Device identification integration: ingest and store device tokens, reuse-detection logic
- SMS OTP provider integration for step-up confirmations
- Bank/UPI penny-drop validation integration: reserved for high-risk escalation path only
- fraud app: rule-based risk scoring engine, CustomerRiskProfile auto-update via signals, DeliveryAgentRiskProfile tracking, outcome-based self-tuning suggestion logic
- Admin dashboard APIs: flagged queue, risk profiles, delivery agent risk view, analytics endpoints, admin action audit log
- Customer notification service: SMS/in-app notification triggered on return approval/rejection

Phase 3 — Integration

- Connect React frontend to Django APIs end-to-end
- Wire the full verification state machine (passive scoring → step-up → escalation → risk scoring)
- Test auto-approve vs flag-for-review paths with sample data

Phase 4 — Merchant Tools & Extension

- Finalize merchant dashboard analytics
- Build the Chrome browser extension (merchant ops tool) once core platform APIs are stable — see Section 11 for scope

Phase 5 — Hardening & Launch Prep

- Security review (device data handling, bank-validation data handling, PII, DPDP Act compliance)
- Rate-limiting / anti-abuse protection on SMS OTP and other verification endpoints to prevent OTP-flooding a customer's phone number
- Performance testing on checkout and return flows
- Pilot with a small D2C brand before wider rollout

10. Open Decisions to Finalize Before Building

- Tenant isolation strategy for the multi-tenant SaaS — shared DB with merchant_id row-scoping (simplest, chosen as default) vs. schema-per-tenant vs. DB-per-tenant (cost/complexity tradeoff as merchant count grows)
- Which SMS OTP provider to use as the step-up channel (cost, delivery/completion rate)
- Bank/UPI penny-drop validation provider selection and consent-flow design (DPDP Act compliance)
- Rule weights/thresholds for the fraud risk engine (needs initial estimates, then tuning with real data)
- Source for wedding/festival calendar data used to auto-tighten seasonal risk thresholds (regional calendar API vs manually maintained list)
- Threshold/process for flagging a delivery agent as a collusion risk, and how that's handled operationally (retraining vs removal from doorstep-proof duty)
- Scope of the extension (who uses it and for what)
- Data retention policy for device data and doorstep proof files (privacy/compliance)

11. Browser Extension — Merchant Ops Tool

A Chrome browser extension (Manifest V3) gives merchants a fast way to triage flagged orders/returns without opening the full admin dashboard. It calls the same admin APIs from Section 6/9 (flagged queue, risk profile, delivery agent risk view, approve/reject), scoped automatically to that merchant's tenant via their API token — no separate backend needed.

Features

- Toolbar badge showing a live count of flagged items awaiting review
- Popup panel: flagged queue list with risk score/tier, customer summary, one-click Approve/Reject
- Quick-glance delivery agent risk indicator on return items, linking through to the full delivery agent risk view in the dashboard
- Desktop notification when a new high-risk order or return is flagged
- Settings page: connect to the merchant's store (API base URL + auth token), configurable polling interval

Tech

- Chrome Extension, Manifest V3 (service worker background script, no persistent background page)
- Calls existing Django REST Framework admin endpoints — flagged queue, risk profile, delivery agent risk view, approve/reject
- Local extension storage for auth token/settings, periodic polling, native notifications for alerts
- Auth: merchant-scoped API token generated from the admin dashboard, entered once in the extension's settings page

Build timing: after Phase 3 integration, once the admin APIs are stable, as scoped in Phase 4.

12. How It Works — End-to-End Flow

Shopper Flow

The shopper signs up (email or Google). Passive device identification starts silently in the background, requiring no action from the shopper. The shopper browses the catalog, adds items to cart, and checks out using COD or prepaid. The risk engine scores the order using order/return history, device signals, and seasonal signals. A low-risk order is placed instantly with zero friction. Later, if the shopper requests a return, the delivery partner captures proof at doorstep pickup (a geo-stamped digital signature and a barcode/QR scan) rather than relying on the shopper's own attestation. A medium-risk return triggers an SMS OTP sent to confirm it's genuine. If confirmed, the return is approved and the shopper is notified by SMS or in-app message. If not confirmed, or if the case is high-risk, it escalates to a bank/UPI penny-drop check and manual review, and the shopper is notified of the outcome once a decision is made.

Merchant Flow

The merchant signs up, creates their store (tenant), and receives an API token. Once the storefront is live, every order and return is scored automatically. Low-risk cases auto-approve with no merchant action needed. Flagged cases land in the Flagged Queue, visible from either the dashboard or the browser extension. The merchant opens a case, reviews the risk score, the customer's risk profile, and delivery-agent risk context if relevant, then approves or rejects it. The action is logged in the audit trail along with who took it and when. Over time, the dashboard surfaces rule-weight tuning suggestions based on which auto-approved cases turned out risky versus which flagged cases were cleared on review; the merchant chooses whether to apply or ignore each suggestion.

Chrome Extension Flow (quick triage)

The merchant installs the extension and enters their API token in Settings, which scopes the extension to only that merchant's tenant data. The toolbar badge shows a live count of pending flagged items, and a desktop notification fires when a new high-risk order or return is flagged. Clicking the badge opens a popup showing the flagged queue, where the merchant can approve or reject with one click — using the same audit trail and risk data as the full dashboard.

13. System Architecture

High-level shape of the system, tying together the components described in the sections above.

Frontend layer

A single React.js application serves both the shopper-facing storefront and the merchant-facing admin dashboard as separate route trees within the same codebase (or as two separate React apps sharing a component library, depending on team preference). The frontend talks to the backend exclusively through the Django REST Framework API — no direct database access from the client.

Backend layer

A Django + Django REST Framework application, organized into the app modules described in Phase 2: accounts, catalog, orders, returns, fraud, and the admin dashboard API surface. All requests pass through a tenant-scoping middleware that resolves the `merchant_id` for the current request (from the authenticated user or API token) and makes it available to every view and query for that request.

Data layer

A single shared PostgreSQL database holds all tenant data, with `merchant_id` scoping enforced at the application layer via the middleware above. Object storage (S3-compatible) holds doorstep return-proof records and any device-session data that doesn't belong in the relational database.

Risk-scoring layer

The fraud app sits alongside the orders and returns apps as an internal service: whenever an order is placed or a return is requested, the relevant app calls into the fraud engine synchronously to get an initial risk classification before the request completes. Follow-up scoring (after OTP or bank verification completes) happens through the same engine, triggered by webhook/callback from the verification provider or by the frontend polling for status.

Asynchronous/background processing

SMS OTP dispatch, bank/UPI penny-drop calls, and customer notifications are handled as background jobs (e.g., via Celery with a Redis or database-backed queue) so the checkout and return-request endpoints stay fast and don't block on third-party API latency.

Integration points

- SMS/OTP provider (e.g., MSG91, Twilio, 2Factor) — step-up confirmation and customer notifications
- Payment gateway / bank-verification provider (e.g., Razorpay, Cashfree) — penny-drop validation
- Festival/wedding calendar data source — feeds seasonal threshold auto-tightening
- Google OAuth — shopper sign-in

Merchant-facing surfaces

The admin dashboard (part of the React frontend) and the Chrome browser extension both consume the same admin API endpoints, scoped by the merchant's API token, so there is a single source of truth for flagged queue state, risk profiles, and audit history regardless of which surface the merchant uses.

ADDENDUM — Flaw Remediation & Revised Design

(Reserve Implementation Plan)

The section below supplements the Implementation Plan above with corrections identified in review. Where the two disagree, this addendum represents the corrected/updated approach.

1. Risk Signals Are Too Simplistic → Weighted, Contextual Scoring

Replace raw counters with rate-based, time-decayed, combined signals. The COD refusal rate should be calculated as refusals divided by total COD orders in the trailing 90 days, not a lifetime count. Return rate should be tracked per category, since a customer who returns 40% of ethnic wear but 0% of electronics is a different case than someone who returns everything. Return timing should be scored relative to category norms — a wedding-outfit return on day 6 is a different signal than one on day 1.

Hard rule: no single signal (device reuse, festival timing, one refusal) can independently push a case to reject or high-risk-block. Every signal is a weighted input into a composite score, and a minimum of two independent signal categories must agree before an account is auto-restricted.

2. Verification Treated as Proof → Verification Is a Risk Modifier, Never a Bypass

SMS OTP and bank/UPI validation confirm who the account belongs to, not whether this specific return is legitimate. Verification status should no longer determine approval directly. A verified identity reduces the risk score by a bounded amount — enough to move a high-risk case down to medium, never straight to auto-approve. Manual review remains reachable for any case above the high-risk threshold, regardless of verification outcome.

3. State Machine Logic Problem → Verification Feeds Back Into Scoring, Doesn't Bypass It

The corrected flow: an initial risk score is computed from passive signals. Based on that score, the appropriate verification step is triggered (none, OTP, or bank/UPI). The verification outcome and its confidence level are then combined with the initial score to produce an updated risk assessment, which is what the decision engine actually acts on. The previous design let a confirmed OTP flow straight to approval; the corrected design always recomputes a score after verification and routes it through the same decision engine, so verification changes the inputs to the decision rather than skipping the decision itself.

4. Device Signal Collection Needs Reduction

Passive signal collection for the MVP should be limited to account age, order/return history, address consistency across orders, a lightweight device-reuse identifier (a client-generated token stored via cookie/local storage, not any form of device fingerprinting library), and payment method pattern (COD vs prepaid mix). Any more invasive session-level tracking is deferred to a later phase, gated on demonstrated value over this reduced signal set, a documented DPDP-compliant consent flow, and a defined retention/deletion policy.

5. Return Reasons Underused → Structured Taxonomy Feeding the Risk Engine

The return reason field should be constrained to a fixed set of values: wrong size, changed mind, damaged product, wrong product received, missing item or component, quality issue, not as described, or other. Each reason carries a base risk weight and interacts with category — wrong size on ethnic wear during wedding season scores differently than wrong size on daily-wear basics, and changed mind on a high-value item from a first-time customer scores higher than damaged product from a repeat, low-risk customer.

6. Delivery-Agent Collusion → Baseline-Adjusted Anomaly Detection, Not Accusation

Replace a raw "return rate per agent" metric with an expected-versus-actual model: the expected return rate for an agent is the baseline return rate for their route/pincode/category mix, and the anomaly score is the gap

between actual and expected. Only a sustained, statistically meaningful deviation — with a defined minimum sample size, such as 30 deliveries, over a rolling window — surfaces the agent in a review queue. That queue is explicitly labeled an investigation signal, never a confirmed-collusion finding, and no automated action (such as removal from doorstep-proof duty) happens without human sign-off.

7. Self-Tuning Underspecified → Recommendation-Only, With Governance

Self-tuning threshold suggestions are read-only in the first release: the dashboard shows suggested changes, but nothing auto-applies. A suggestion only appears once a minimum sample size (e.g. 50 outcomes per rule since the last change) and a minimum evaluation window (e.g. 14 days) have been met. Every applied change is logged with its before/after values, who approved it, and when, feeding the same audit log used elsewhere in the system. A rollback option restores the previous threshold set in one step. Fully automatic, unattended threshold adjustment stays out of scope until the platform has enough historical outcome data to validate the suggestion engine itself.

8. Fraud Outcome Labels Too Coarse → Expanded Set

The three-value outcome field should be replaced with a richer set: auto-approved, confirmed fraud, suspected fraud, legitimate return, logistics issue, merchant issue, insufficient evidence, and verification abandoned. This distinction is what makes both the self-tuning suggestions (Section 7) and the delivery-agent anomaly detection (Section 6) trustworthy — thresholds can't be tuned on outcomes that don't separate "this was fraud" from "the courier lost the item."

9. Data Model Too Simplified → Normalize History, Don't Overwrite It

The order's items should be broken out into their own records (an `OrderItem` per line, holding product, quantity, and price) rather than a single combined field. Each risk-scoring event should be preserved as its own historical record — including the score, tier, which rule version produced it, and which signals were used — rather than overwriting a single `risk_score` field on the order or return. Each verification attempt (device-only, SMS OTP, bank-validated, or doorstep proof) should likewise be preserved as its own event record with its method, status, and confidence weight, rather than collapsing verification history into a single status field. Returns should be broken into individual return-line records so partial-order returns are representable, and a timeline of return-related events (requested, pickup scheduled, doorstep proof captured, OTP sent, OTP confirmed, escalated, reviewed, decided) should be kept for each return request. Preserving this history — rather than repeatedly overwriting a handful of summary fields — is also what makes rule versioning possible, since you can trace exactly which threshold set produced any historical score, which the self-tuning audit trail in Section 7 depends on.

10. Extension Auth Is a Security Risk → Least-Privilege, Short-Lived, Revocable

The long-lived static API token pasted into the extension's settings should be replaced with a short-lived token (for example, valid for one hour) that refreshes through a login flow rather than being permanently stored as a secret. The extension should operate under a scoped role — able to read the flagged queue and risk views, and approve or reject cases — but without full merchant-admin API access. Every extension request should be re-authorized server-side against the merchant's tenant, rather than trusting the client-held token alone. High-impact actions taken from the extension, such as approving or rejecting a high-risk case, should require a confirmation step rather than resolving instantly on a single click. The merchant should be able to revoke all extension sessions from the main dashboard at any time.

11. Multi-Tenant Isolation Needs a Database-Level Boundary

The existing `merchant_id` middleware scoping should be kept, but backed by PostgreSQL Row-Level Security policies on every tenant-scoped table, keyed off a per-request session variable identifying the current merchant. This means that even a missing or incorrect `merchant_id` filter in application code cannot leak another tenant's

rows — the database itself refuses to return them. Automated authorization tests that specifically attempt cross-tenant reads, and assert they return nothing, should be part of the standard test suite.

12. MVP Scope Too Large → Trimmed First Release

The first release should include only the core storefront (catalog, cart, checkout for COD and prepaid, order history), the reduced-signal risk scoring described in Section 4 above, SMS OTP step-up, doorstep QR/barcode proof capture (without bank/UPI validation), and the merchant review dashboard (flagged queue, approve/reject, audit log). Bank/UPI penny-drop validation, delivery-agent anomaly detection, self-tuning suggestions, and the Chrome extension should all be deferred to a later phase, so the core fraud workflow can be validated with real merchant data before adding surface area that's expensive to secure and hard to justify without evidence it helps.

Dashboard UI Appendix

Supplement to: AI-Assisted Return-Fraud Detection E-Commerce Platform — Implementation Plan

Scope note: The original PDF defines the fraud, order, return, merchant-review, and data workflows. This appendix adds the requested dashboard navigation and page concepts. Billing & Invoice is a UI/business module interpretation and is not explicitly defined in the source plan.

14. Merchant Dashboard Navigation & Page Flow

The following screens add the requested frontend information architecture. Functional content follows the implementation plan; exact visual layouts are UI interpretations. The core merchant workflow remains flagged queue → contextual review → approve or reject → audit trail.

14.1 Main Merchant Dashboard

LOGO / PLATFORM	Search	Notifications	Admin	
Dashboard	Dashboard Overview			
Orders	Total	Flagged	Returns Pending	
Customers	Orders	Cases	Review	
Returns				
Flagged Cases	Recent Flagged Cases			
Analytics	Customer	Order	Risk	Status
Audit Log	User A	#1024	HIGH	Review
Billing & Invoices	User B	#1025	MED	Review
Settings				

14.2 Orders Page

Dashboard / Orders	Search	Alerts	
Dashboard	Orders	[All] [COD] [Prepaid] [Flagged]	
Orders ←			
Customers	Order ID	Customer	Payment
Returns	#1024	User A	COD
Flagged Cases	#1025	User B	COD
Analytics	#1026	User C	Prepaid
Audit Log			
Billing & Invoices	Click an order →	Order Details	
Settings			

14.3 After Clicking One Order — Order Details

← Back to Orders	
Order #1025	
Customer Information	
Customer Name / Account / Address	
Order Details	
Products / Quantity / Total	
Payment Method: COD or Prepaid	
Order Status	
Risk Information	
Risk Tier	
Verification Status / Method	
Relevant Risk Context	
[View Customer Profile] [View Related Return]	

14.4 After Clicking a Specific Customer — Customer Risk Profile

← Back to Customers	
CUSTOMER PROFILE	
Customer / Account	CURRENT RISK: Risk Tier
[Total Orders] [Total Returns] [COD Refusals]	
Order & Return History	
Date Order Payment Status Risk	
Risk & Verification History	
• Risk scoring events • Signals used • Verification attempts	
• Return-related timeline	

14.5 Billing & Invoice Dashboard Integration

Billing & Invoices

Current Plan / Subscription

Status

[Manage Plan]

[Current Billing]

[Next Billing]

Recent Invoices

Invoice ID	Date	Amount	Payment Status	Action
INV-001	...	...	Paid	View / Download
INV-002	...	...	Paid	View / Download

14.6 Recommended Navigation Relationship

- Merchant Dashboard
- Dashboard
 - Orders
 - Orders List
 - Order Details → Customer Profile / Related Return
 - Customers
 - Customers List
 - Customer Risk Profile
 - Returns / Flagged Cases → Review → Approve / Reject → Audit Log
 - Analytics
 - Audit Log
 - Billing & Invoices (optional module; not defined in original plan)
 - Settings

15. Change Assessment

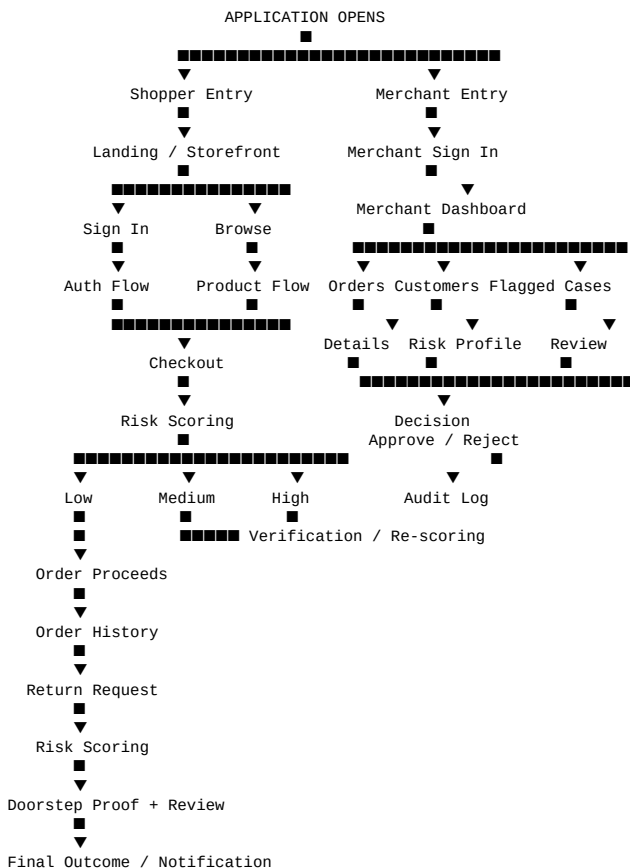
No changes are made to the original fraud-detection logic, verification state machine, revised data model, security remediation, or trimmed MVP scope. The added dashboard pages are a frontend information-architecture appendix. Billing & Invoice should remain clearly marked as an optional future/business module unless the implementation plan is formally expanded.

Complete Application UI, Navigation & Architecture Appendix

Supplement to the AI-Assisted Return-Fraud Detection E-Commerce Platform Implementation Plan

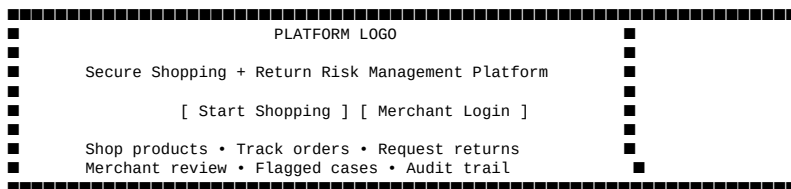
Document basis. This appendix follows the implementation plan and its revised Addendum. The Addendum takes priority where it changes the earlier plan. The exact appearance of screens is not specified in the source PDF; therefore, screen layouts and navigation presentation below are UI/UX interpretations of the documented features. Billing & Invoices remains an optional business module because it is not defined in the source MVP.

16. Complete Application Journey — Starting to Ending



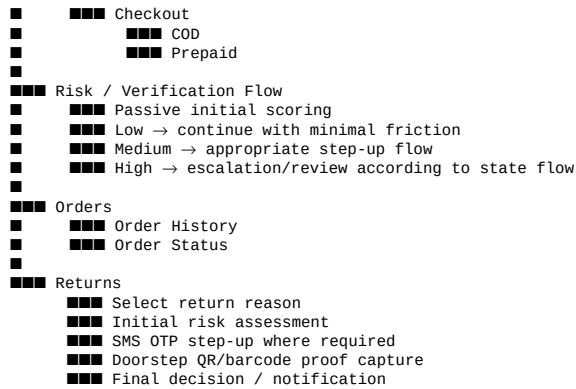
17. Opening Interface

The source plan defines registration/login, catalog browsing, merchant onboarding, and separate shopper and merchant functionality. A practical opening screen can therefore provide two clear entry paths: **Shop** and **Merchant/Admin**.

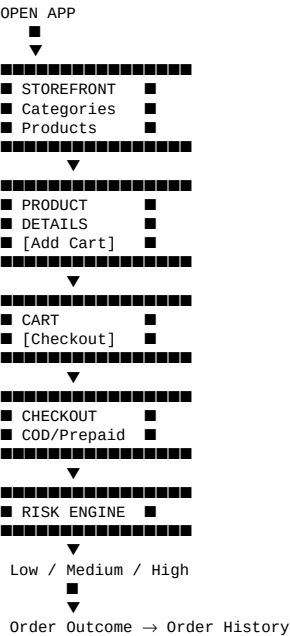


18. Shopper-Side Interface Architecture

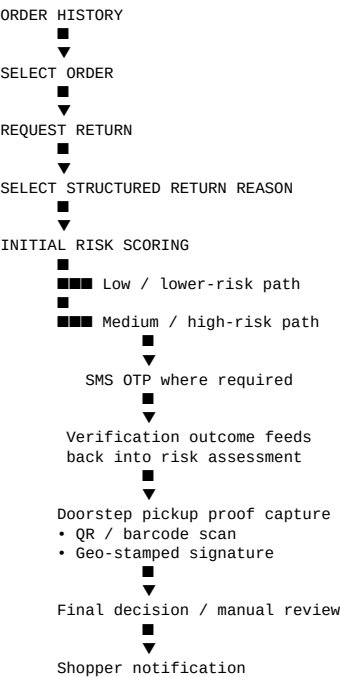




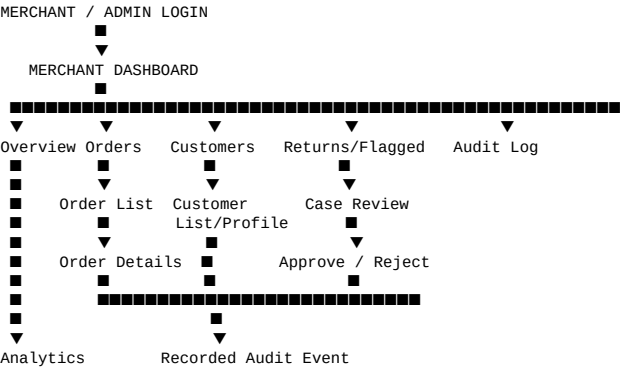
18.1 Shopper Screen Sequence



19. Return Journey — End-to-End

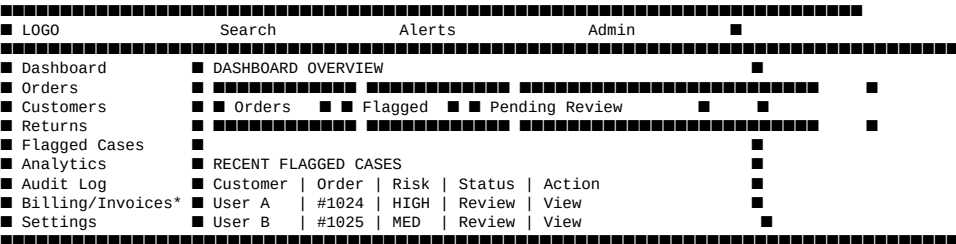


20. Merchant/Admin Interface Architecture



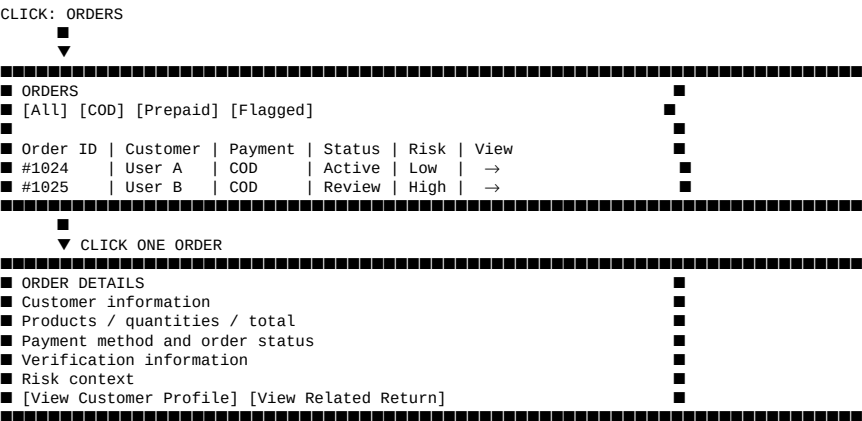
Optional UI/business module:
Billing & Invoices → Current Plan → Billing → Invoice History

20.1 Main Merchant Dashboard

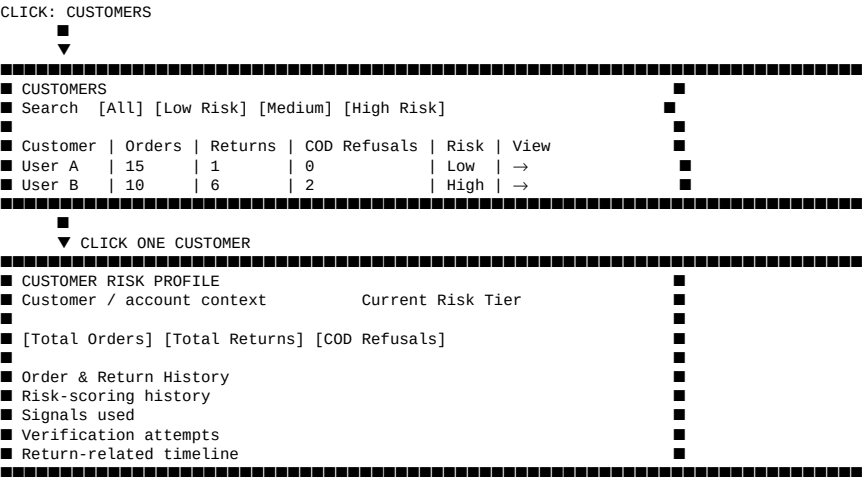


* Optional module; not defined in source MVP.

21. Orders Navigation

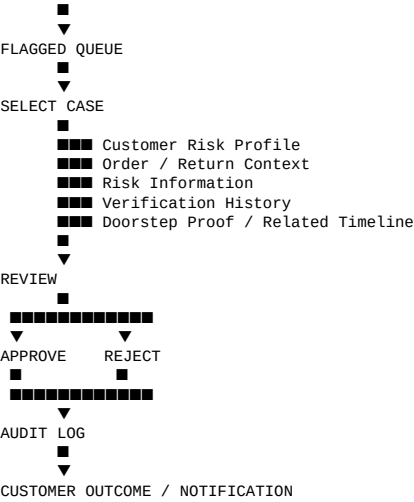


22. Customers Navigation



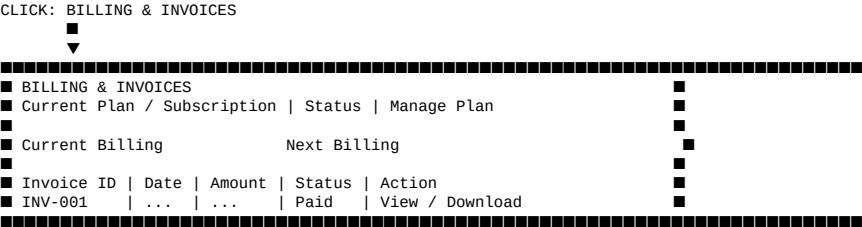
23. Flagged Case Review Flow

FLAGGED CASES

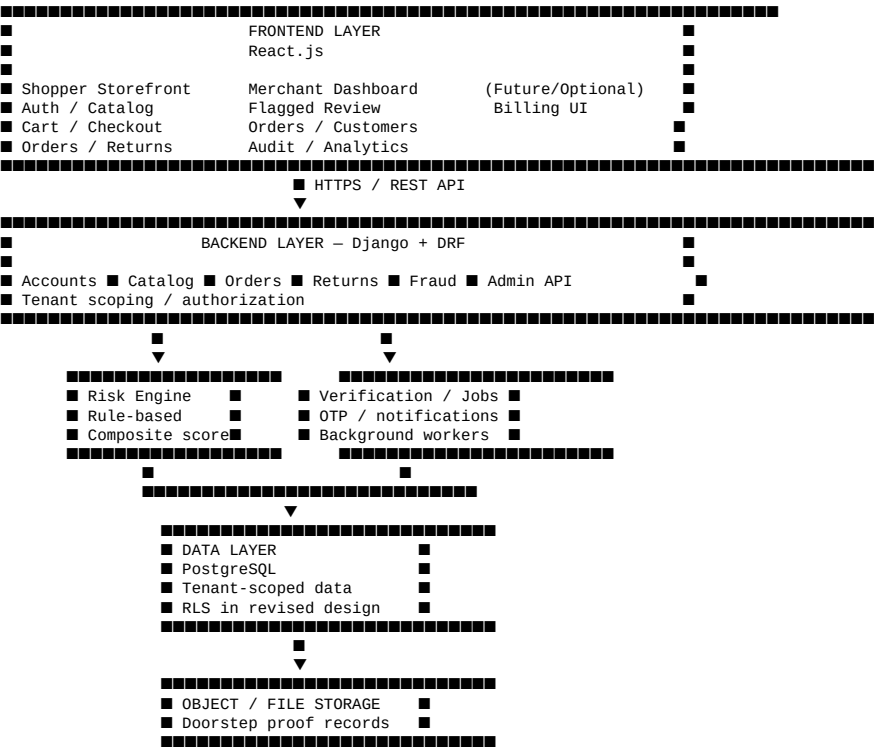


24. Billing & Invoice Navigation

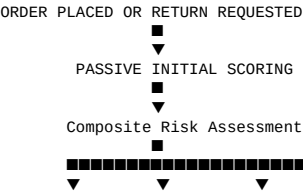
This section is retained from the previous dashboard appendix as an optional UI/business module. It is not part of the source PDF's defined MVP.

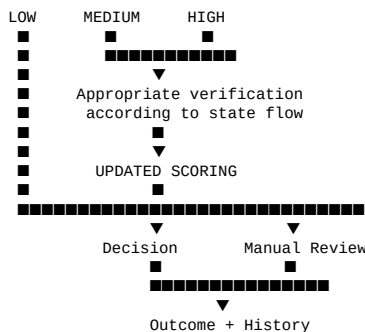


25. System Architecture — Application to Backend



26. Core Risk & Verification Architecture





27. MVP Boundary in the Final UI

The revised Addendum trims the first release to the core storefront, COD/prepaid checkout, reduced-signal risk scoring, SMS OTP step-up, doorstep QR/barcode proof capture, and the merchant review dashboard with approve/reject and audit log. Bank/UPI penny-drop validation, delivery-agent anomaly detection, self-tuning suggestions, and the Chrome extension are deferred. The UI architecture should therefore visually distinguish MVP screens from later-phase screens.

28. Final Application Map

